

1. **Oracle and FreeSQL:** Practising on Oracle with FreeSQL · Oracle compared with other SQL dialects
2. **SQL languages:** The sub-languages of SQL · DDL: defining the schema · DML: reading and changing rows · DCL: users, roles and privileges · TCL: transactions
3. **Constraints:** PRIMARY KEY, UNIQUE and NOT NULL · FOREIGN KEY and referential actions · CHECK and DEFAULT
4. **Indexes:** What an index is · Composite and covering indexes · What indexes cost · Kinds of index
5. **Clustering:** Clustered and non-clustered indexes · Table clusters in Oracle (and CLUSTER in PostgreSQL)
6. **Partitioning:** Horizontal partitioning · Partition pruning and maintenance · Vertical partitioning and sharding
7. **Efficiency:** Reading a query plan · Conditions that can use an index · Writing efficient queries

Oracle and FreeSQL

Practising on Oracle with FreeSQL

FreeSQL (freesql.com) is Oracle's free online worksheet: you write Oracle SQL in the browser and run it on a real Oracle database, with nothing to install. It is where you practise in this course.

- Run one statement: put the cursor in it and run. Run the whole script: use the script option of the worksheet.
- Your schema persists between sessions: drop what you no longer need.
- The simulation here does not support PL/SQL, partitioning, sequences or privileges: for those, FreeSQL is the only place to run them.

Common mistake. Trusting only the simulation. It translates a subset of Oracle to a different engine: when its result surprises you, check it in FreeSQL.

Oracle compared with other SQL dialects

SQL is a standard, but every DBMS has its own dialect. Most errors of students who learned SQL elsewhere come from a few differences: row limiting, types, dates, empty strings and transactions.

Oracle versus SQLite, PostgreSQL and MySQL

Topic	Oracle	Others
First n rows	FETCH FIRST n ROWS ONLY	LIMIT n
Set difference	MINUS	EXCEPT
Text type	VARCHAR2(n), CLOB	TEXT, VARCHAR(n)
Numbers	NUMBER(p, s)	INTEGER, NUMERIC, REAL
Auto number	GENERATED ALWAYS AS IDENTITY	AUTOINCREMENT, SERIAL, AUTO_INCREMENT
Current date	SYSDATE	date('now'), CURRENT_DATE, NOW()
Date literal	DATE '2024-01-31', TO_DATE(...)	A plain string
Null replacement	NVL(a, b), COALESCE	IFNULL(a, b)
Empty string	Is NULL	Is an empty string
Select without table	SELECT 1 FROM dual	SELECT 1
Transactions	Start with the first DML; DDL commits	BEGIN / START TRANSACTION
Data dictionary	USER_TABLES, USER_TAB_COLUMNS	sqlite_master, information_schema
Execution plan	EXPLAIN PLAN FOR + DBMS_XPLAN	EXPLAIN

Common mistake. Writing a comparison like `WHERE signup_date >= '2024-01-01'` against a DATE column. It relies on the session's date format and often fails (ORA-01861); write `DATE '2024-01-01'`.

SQL languages

The sub-languages of SQL

SQL is one language with several sub-languages, grouped by what a statement does: define the structure, work

with the data, control who can do what, and group changes into transactions.

The four groups

Sub-language	Statements	What it acts on
DDL · data definition	CREATE, ALTER, DROP, TRUNCATE, RENAME	The schema: tables, columns, constraints, indexes, views
DML · data manipulation	SELECT, INSERT, UPDATE, DELETE, MERGE	The rows stored in the tables
DCL · data control	GRANT, REVOKE	Users, roles and their privileges
TCL · transaction control	BEGIN, COMMIT, ROLLBACK, SAVEPOINT	When changes become permanent or are undone

Example. CREATE TABLE student (...) is DDL; INSERT INTO student VALUES (...) is DML; GRANT SELECT ON student TO teacher is DCL; COMMIT is TCL.

Common mistake. Thinking TRUNCATE is the same as DELETE without WHERE. TRUNCATE is DDL: it empties the table at once, usually cannot be filtered, and in many DBMSs commits implicitly.

DDL: defining the schema

CREATE builds an object, **ALTER** changes it and **DROP** removes it, data included. DDL turns the logical model into real tables.

- Oracle types: VARCHAR2(n) for text, NUMBER(p, s) for every number, DATE (it also stores the time), TIMESTAMP, CLOB, BLOB. From 23ai there is also BOOLEAN.
- Auto-numbered keys use GENERATED ALWAYS AS IDENTITY; there is no AUTOINCREMENT.
- Each DDL statement updates the data dictionary, which you can query: USER_TABLES, USER_TAB_COLUMNS, USER_INDEXES, USER_CONSTRAINTS.

Example you can run on the website

```
CREATE TABLE course (
  course_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title     VARCHAR2(40) NOT NULL,
  credits   NUMBER(3,1) NOT NULL
);
ALTER TABLE course ADD (semester NUMBER(1));
INSERT INTO course (title, credits, semester) VALUES ('Databases', 6, 1);
SELECT * FROM course;
SELECT table_name FROM user_tables;
DROP TABLE course;
SELECT table_name FROM user_tables;
```

Common mistake. Dropping and re-creating a table to change one column. ALTER TABLE keeps the rows; DROP loses them.

DML: reading and changing rows

SELECT reads, **INSERT** adds rows, **UPDATE** changes them and **DELETE** removes them. UPDATE and DELETE affect **every row** that matches the WHERE.

- Joins follow the foreign keys: enrolment.course_id = course.course_id.
- The runner shows how many rows each INSERT, UPDATE or DELETE changed.
- To keep only the first rows of a result use FETCH FIRST n ROWS ONLY; Oracle has no LIMIT.

Example you can run on the website

```
INSERT INTO enrolment VALUES ('S03', 'C20', 6.0);
UPDATE enrolment SET grade = grade + 0.5 WHERE course_id = 'C10' AND grade < 6;
DELETE FROM enrolment WHERE student_id = 'S02' AND course_id = 'C11';
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course c ON c.course_id = e.course_id
ORDER BY s.name, c.title;
SELECT c.title, COUNT(*) AS students, ROUND(AVG(e.grade), 2) AS average
FROM enrolment e JOIN course c ON c.course_id = e.course_id
```

```
GROUP BY c.title
HAVING COUNT(*) >= 2;
```

Example. The average grade per course is `SELECT course_id, AVG(grade) FROM enrolment GROUP BY course_id.`

Common mistake. Filtering an aggregate with `WHERE` (`WHERE AVG(grade) > 7`). `WHERE` runs before grouping; use `HAVING AVG(grade) > 7`.

DCL: users, roles and privileges

GRANT gives a privilege on an object to a user or role; **REVOKE** takes it away. Give each role only what it needs: the **least privilege principle**.

- `WITH GRANT OPTION` lets the receiver grant the same privilege to others; use it sparingly.
- `REVOKE` removes the privilege from that role; members lose it unless another role still grants it.
- Oracle names objects as `schema.table`: `GRANT SELECT ON hr.enrolment TO teacher` refers to the table of the schema `hr`.

Oracle · [not runnable here](#)

```
CREATE ROLE teacher;
CREATE ROLE reporting;
CREATE USER ana IDENTIFIED BY "change-me";
GRANT CREATE SESSION TO ana;
GRANT teacher TO ana;

GRANT SELECT, INSERT, UPDATE ON enrolment TO teacher;
GRANT SELECT ON student TO teacher;
GRANT SELECT ON course TO teacher;

CREATE VIEW course_average AS
  SELECT course_id, AVG(grade) AS average FROM enrolment GROUP BY course_id;
GRANT SELECT ON course_average TO reporting;      -- no access to individual grades

REVOKE UPDATE ON enrolment FROM teacher;
```

Common mistake. Letting the application connect as the database owner. A bug or an SQL injection can then drop tables; with a role limited to DML on its tables, it cannot.

TCL: transactions

A **transaction** groups statements that must succeed or fail together. **COMMIT** makes them permanent, **ROLLBACK** undoes them, and a **SAVEPOINT** marks a point you can roll back to.

- `SAVEPOINT name ... ROLLBACK TO SAVEPOINT name` undoes only the work after the savepoint.
- Keep transactions short: while one is open, it may hold locks that make others wait.
- Oracle readers never block writers (it keeps old versions of rows), and by default a query sees the data committed when the query started.

Example you can run on the website

```
DELETE FROM enrolment WHERE student_id = 'S03' AND course_id = 'C11';
INSERT INTO enrolment VALUES ('S03', 'C20', NULL);
SAVEPOINT before_grades;
UPDATE enrolment SET grade = 10;          -- oops: every row
ROLLBACK TO SAVEPOINT before_grades;
COMMIT;
SELECT * FROM enrolment WHERE student_id = 'S03';
DELETE FROM enrolment;
SELECT COUNT(*) AS rows_inside_transaction FROM enrolment;
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM enrolment;
```

Example. The second block changes a grade by mistake and rolls back to the savepoint: the move between courses is kept, the wrong grade is not.

Common mistake. Running DDL in the middle of a transaction and expecting to roll back. A DDL statement commits everything before it, so the earlier changes can no longer be undone.

Constraints

PRIMARY KEY, UNIQUE and NOT NULL

The **primary key** identifies each row: unique and never NULL. **UNIQUE** forbids duplicates in other candidate keys, and **NOT NULL** makes a column mandatory.

- Composite key: PRIMARY KEY (student_id, course_id).
- The last statement of the example fails on purpose: read the error message.

Example you can run on the website

```
INSERT INTO student VALUES ('S04', 'Marta Paz', NULL); -- a second NULL e-mail is allowed
SELECT * FROM student;
INSERT INTO student VALUES ('S05', 'Ana Ruiz', 'ana@uni.es'); -- duplicate e-mail
```

Common mistake. Using only a surrogate id and forgetting the natural key. With id as PK and no UNIQUE (email), the same student can be inserted twice.

FOREIGN KEY and referential actions

A **foreign key** only accepts values that exist in the referenced key. Its **referential action** says what happens to child rows when the parent row is deleted or its key changes.

- CASCADE fits parts that make no sense alone (an enrolment without its student, an order line without its order).
- No action (the default) protects reference data (a course, a department).
- Inserting a child with a missing parent fails with ORA-02291 (parent key not found); deleting a parent with children fails with ORA-02292 (child record found).

Example you can run on the website

```
DELETE FROM student WHERE student_id = 'S01'; -- CASCADE: Ana's enrolments go too
SELECT * FROM enrolment ORDER BY student_id;
DELETE FROM course WHERE course_id = 'C10'; -- C10 still has enrolments: rejected
```

Common mistake. Using CASCADE everywhere. Deleting one department could then silently delete its courses and all their enrolments.

CHECK and DEFAULT

CHECK states a condition each row must meet, such as a grade between 0 and 10. **DEFAULT** gives a column its value when the INSERT does not say one.

- A CHECK passes when the condition is true or unknown (NULL); add NOT NULL if the value is required.

Example you can run on the website

```
CREATE TABLE loan (
  loan_id  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  book     VARCHAR2(60) NOT NULL,
  loan_date DATE DEFAULT SYSDATE NOT NULL,
  days     NUMBER(2) DEFAULT 15 NOT NULL,
  status   VARCHAR2(10) DEFAULT 'open' NOT NULL,
  CONSTRAINT days_range CHECK (days BETWEEN 1 AND 60),
  CONSTRAINT status_values CHECK (status IN ('open', 'returned', 'lost'))
);
INSERT INTO loan (book) VALUES ('Don Quixote');
INSERT INTO loan (book, days, status) VALUES ('Ficciones', 30, 'returned');
```

```
SELECT * FROM loan;
INSERT INTO loan (book, days) VALUES ('Hopscotch', 90); -- breaks days_range
```

Common mistake. Checking ranges only in the application. Data loaded with a script or another program then skips the rule.

Indexes

What an index is

An **index** is a separate, sorted structure (usually a **B+ tree**) that maps the values of some columns to their rows. With it, the DBMS can **search** instead of **scanning** the whole table.

- The primary key and each UNIQUE constraint already have an index.
- Foreign keys do **not** get one automatically in Oracle; index them if you join or filter by them.

Example you can run on the website

```
EXPLAIN PLAN FOR SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT * FROM customer WHERE email = 'user43210@mail.com';
CREATE INDEX idx_customer_email ON customer (email);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT * FROM customer WHERE email = 'user43210@mail.com';
```

Common mistake. Thinking an index changes the result. It only changes how fast the DBMS finds the rows; the query returns exactly the same data.

Composite and covering indexes

A **composite** index sorts by several columns, in order. It helps queries that filter on its **leftmost** columns. A **covering** index contains every column a query needs, so the table is not read at all.

- Leftmost-prefix rule: (a, b, c) serves filters on a, a, b and a, b, c.
- One composite index can replace several single-column ones.

Example you can run on the website

```
CREATE INDEX idx_city_date ON customer (city, signup_date);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE city = 'Madrid' AND signup_date >= DATE '2023-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE signup_date >= DATE '2023-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT city, signup_date FROM customer WHERE city = 'Sevilla';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT COUNT(*) AS madrid_since_2023 FROM customer WHERE city = 'Madrid' AND signup_date >= DATE '2023-01-01';
```

Common mistake. Creating (signup_date, city) for queries that always fix the city and give a date range. With the range first, the second column cannot narrow the search.

What indexes cost

Indexes speed up reads but slow down writes: every INSERT, UPDATE or DELETE must also update **each index** of the table. They also take disk and memory.

When to index a column

Usually yes	Usually no
Foreign keys used in joins	Columns never used in WHERE, JOIN or ORDER BY
Columns filtered with = or ranges that return few rows	Low-selectivity columns (gender, yes/no)
Columns used to sort large results (ORDER BY ... LIMIT)	Small tables that fit in a few blocks
Search keys of frequent queries	Tables with heavy writes and rare reads (logs)

Example you can run on the website

```
INSERT INTO plain_log
SELECT LEVEL, MOD(LEVEL, 997), 'action' || MOD(LEVEL, 13), DATE '2024-01-01' + MOD(LEVEL, 365)
FROM dual CONNECT BY LEVEL <= 20000;
INSERT INTO indexed_log
SELECT LEVEL, MOD(LEVEL, 997), 'action' || MOD(LEVEL, 13), DATE '2024-01-01' + MOD(LEVEL, 365)
FROM dual CONNECT BY LEVEL <= 20000;
SELECT index_name AS indexes_on_indexed_log FROM user_indexes WHERE table_name = 'INDEXED_LOG';
```

Common mistake. Indexing every column "just in case". Each index slows every write and the optimizer still uses only the useful ones.

Kinds of index

The **B-tree** is the default and serves equality, ranges and sorting. Other kinds solve specific problems: **hash** for equality only, **bitmap** for few distinct values, **GIN** for values inside a document or array, **BRIN** for huge, naturally ordered tables.

Main index kinds

Kind	Good for	Not for
B-tree / B+ tree	=, <, >, BETWEEN, ORDER BY, prefix LIKE 'abc%'	Searching inside text or arrays
Hash	Equality only (=) (PostgreSQL, MySQL MEMORY)	Ranges and sorting
Bitmap	Columns with few distinct values in read-mostly tables (data warehouses)	Tables with many concurrent writes
Reverse key (Oracle)	Monotonic keys (sequence numbers) inserted by many sessions at once: spreads the hot right-hand leaf	Range scans: it destroys the key order
GIN / inverted	Full-text search, JSON keys, array elements	Simple scalar columns
BRIN	Very large tables ordered by insertion (dates in a log) (PostgreSQL)	Randomly ordered data

Oracle · not runnable here

```
CREATE INDEX orders_customer_idx ON orders (customer_id);           -- B-tree (default)
CREATE BITMAP INDEX customer_city_bix ON customer (city);         -- few distinct values, data warehouse
CREATE INDEX orders_id_rev ON orders (order_id) REVERSE;          -- reverse-key
CREATE INDEX customer_email_ci ON customer (LOWER(email));         -- function-based
CREATE UNIQUE INDEX open_order_uq ON orders (CASE WHEN status = 'OPEN' THEN customer_id END); -- "partial" unique
```

Common mistake. Creating a bitmap index on a table that many sessions update at once. Each change locks a whole range of rows in the bitmap, so writers wait for each other.

Clustering

Clustered and non-clustered indexes

In a **clustered** index the table rows themselves are stored in the order of the key, inside the index. A table can have only **one**. Every other index is **non-clustered**: it stores the key plus a pointer to the row.

- Choose a clustering key that is short, does not change, and matches frequent range queries.
- Random keys (such as random UUIDs) scatter inserts all over a clustered index; increasing keys append at the end.

Example you can run on the website

```
CREATE TABLE enrolment_by_course (
  course_id VARCHAR2(3),
  student_id VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (course_id, student_id)
) ORGANIZATION INDEX;
INSERT INTO enrolment_by_course VALUES ('C20', 'S01', 7.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S03', 5.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S01', 8.5);
INSERT INTO enrolment_by_course VALUES ('C11', 'S02', 9.0);
```

```
INSERT INTO enrolment_by_course VALUES ('C10', 'S02', 6.5);
SELECT * FROM enrolment_by_course;           -- stored in key order (without ORDER BY the order is never
guaranteed)
EXPLAIN PLAN FOR SELECT * FROM enrolment_by_course WHERE course_id = 'C10';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Common mistake. Confusing a **clustered index** with a **database cluster**. The second is a group of servers working together (replication, high availability), an unrelated meaning of the word.

Table clusters in Oracle (and CLUSTER in PostgreSQL)

An Oracle **table cluster** stores the rows of several tables that share a key (a department and its employees) in the same blocks, which speeds up joins on that key. PostgreSQL has no such thing: its **CLUSTER** command rewrites a table once, in the order of an index.

Oracle · not runnable here

```
CREATE CLUSTER dept_cluster (dept_id VARCHAR2(3));
CREATE INDEX dept_cluster_idx ON CLUSTER dept_cluster;           -- a cluster needs its index

CREATE TABLE department (
  dept_id VARCHAR2(3) PRIMARY KEY,
  name     VARCHAR2(30) NOT NULL
) CLUSTER dept_cluster (dept_id);

CREATE TABLE course (
  course_id VARCHAR2(3) PRIMARY KEY,
  title     VARCHAR2(40) NOT NULL,
  dept_id   VARCHAR2(3) NOT NULL REFERENCES department (dept_id)
) CLUSTER dept_cluster (dept_id);                               -- same blocks as its department
```

Common mistake. Clustering tables that are updated all the time or often read alone. Clusters only pay off for stable data that is read together.

Partitioning

Horizontal partitioning

Partitioning splits one big table into smaller pieces, the **partitions**, according to a **partition key**. Queries still use the single table name; the DBMS sends each row to its partition.

Choosing the method

Method	Typical key	Use it when
Range	Date, id ranges	Queries and data retention go by time
List	Country, region, status	A few known values separate the data
Hash	Customer id	You need even sizes and there is no natural range

Oracle · not runnable here

```
CREATE TABLE orders (
  order_id   NUMBER(10),
  customer_id NUMBER(10) NOT NULL,
  order_date DATE NOT NULL,
  total      NUMBER(10,2) NOT NULL
)
PARTITION BY RANGE (order_date) (
  PARTITION p2025_01 VALUES LESS THAN (DATE '2025-02-01'),
  PARTITION p2025_02 VALUES LESS THAN (DATE '2025-03-01'),
  PARTITION p_max   VALUES LESS THAN (MAXVALUE)           -- rows outside every range
);

-- Oracle creates the monthly partitions by itself:
-- PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH')) (PARTITION p_first VALUES LESS THAN
(DATE '2025-01-01'))
```

```
CREATE INDEX orders_customer_ix ON orders (customer_id) LOCAL; -- one index segment per partition
```

Common mistake. Partitioning a small table. Below many millions of rows, a good index is usually simpler and just as fast.

Partition pruning and maintenance

When the **WHERE** fixes the partition key, the optimizer reads only the matching partitions: **partition pruning**. Old data can then be removed by dropping a whole partition instead of deleting millions of rows.

- Choose the partition key from the most frequent filters.
- A **local** index has one segment per partition and is kept valid when you drop a partition; a unique index must include the partition key to be local.

Oracle · [not runnable here](#)

```
EXPLAIN PLAN FOR SELECT SUM(total) FROM orders
WHERE order_date >= DATE '2025-02-01' AND order_date < DATE '2025-03-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- | Id | Operation                | Name | Pstart | Pstop |
-- | 2 | PARTITION RANGE SINGLE    |      | 2      | 2      | <- only one partition is read
-- | 3 | TABLE ACCESS FULL       | ORDERS | 2      | 2      |

ALTER TABLE orders DROP PARTITION p2025_01 UPDATE INDEXES; -- remove a whole month at once
ALTER TABLE orders EXCHANGE PARTITION p2025_01 WITH TABLE orders_2025_01_archive; -- ...or swap it out as a table
```

Common mistake. Partitioning by a column the queries rarely filter on. Without pruning, every query visits every partition.

Vertical partitioning and sharding

Vertical partitioning splits the **columns** of a table into two tables with the same key. **Sharding** splits the **rows** across different **servers**, each with its own database.

Three ways to split

Technique	Splits	Where the pieces live	Main cost
Horizontal partitioning	Rows	Same database	Queries without the key visit every partition
Vertical partitioning	Columns	Same database	A join to read the whole row
Sharding	Rows	Different servers	Cross-shard joins and transactions

Common mistake. Sharding before it is needed. Indexes, partitioning, read replicas and a bigger server solve most problems with far less complexity.

Efficiency

Reading a query plan

SQL says **what** you want; the **optimizer** decides **how**: which index, which join order, which algorithm. **EXPLAIN PLAN** shows that plan, and the plan is where slow queries are understood.

- TABLE ACCESS FULL = read every row; INDEX RANGE SCAN / INDEX UNIQUE SCAN = go down an index.
- SORT ORDER BY = an extra sort; an index in that order avoids it.

Example you can run on the website

```
EXPLAIN PLAN FOR
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
CREATE INDEX idx_orders_customer ON orders (customer_id);
```

```
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'ORDERS');
EXPLAIN PLAN FOR
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
```

Common mistake. Optimizing by intuition. Read the plan first: the slow part is often not where you expect.

Conditions that can use an index

An index on a column only helps when the condition compares the **bare column**. Wrapping it in a function or calculation hides it from the index: the query scans.

Example you can run on the website

```
EXPLAIN PLAN FOR SELECT COUNT(*) FROM customer WHERE TO_CHAR(signup_date, 'YYYY') = '2023';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT COUNT(*) FROM customer WHERE signup_date >= DATE '2023-01-01' AND signup_date < DATE
'2024-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE LOWER(email) = 'user77@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
CREATE INDEX idx_email_lower ON customer (LOWER(email));
EXPLAIN PLAN FOR SELECT * FROM customer WHERE LOWER(email) = 'user77@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Common mistake. Adding an index and assuming the query will use it. Check the plan; a function on the column is enough to make the index useless.

Writing efficient queries

Ask the database for exactly what you need, in **one set-based statement**, and let it do the work near the data: filter, join and aggregate in SQL, not row by row in the application.

- Select only the columns you use; `SELECT *` reads and sends more data and stops covering indexes from helping.
- Use `EXISTS` to ask "is there any?" instead of `COUNT(*) > 0`: it can stop at the first match.
- Paginate with `ORDER BY ... FETCH FIRST n ROWS ONLY (OFFSET m ROWS to skip)`, ideally on an indexed column.
- Join and filter on indexed keys, and keep transactions short.

Example you can run on the website

```
-- "Has customer 42 ordered anything?": EXISTS can stop at the first order
SELECT CASE WHEN EXISTS (SELECT 1 FROM orders WHERE customer_id = 42) THEN 'yes' ELSE 'no' END AS has_orders FROM
dual;
-- One set-based statement instead of a loop over every order
UPDATE orders SET total = ROUND(total * 0.9, 2) WHERE order_date < DATE '2023-02-01';
COMMIT;
-- Only the columns and rows needed, aggregated in the database
SELECT c.city, COUNT(*) AS orders, ROUND(SUM(o.total), 2) AS spent
FROM orders o JOIN customer c ON c.customer_id = o.customer_id
WHERE o.order_date BETWEEN DATE '2024-01-01' AND DATE '2024-12-31'
GROUP BY c.city
ORDER BY spent DESC
FETCH FIRST 5 ROWS ONLY;
```

Example. The last query of the example answers "top 5 cities by spending in 2024" with one statement, instead of reading every order into the program.

Common mistake. Fetching a whole table to filter it in the application. The database can use indexes and send only the matching rows; the program can do neither.